

COMSATS University Islamabad, Lahore Campus



***OBJECT-ORIENTED PROGRAMMING
THEORY ASSIGNMENT NO.01***

Object Oriented Analysis of a Smart Campus Allocation System

SUBMITTED BY: FARYAL SARFRAZ

SP26-BAI-019

SUBMITTED TO: DR. MUHAMMAD SHAHID BHATTI

BS ARTIFICIAL INTELLIGENCE

SP26-BAI-A (Semester 02)

CSC241

Student-Specific Condition

Some equipment is movable and scarce, so the room and the equipment may need separate allocation decisions.

This condition is integrated into every task and marked [RL (Resource Limitation)] where it shapes a decision.

Task 1

Stakeholder And Requirement Analysis

- Identify at least four stakeholders and state one need for each.

Stakeholder	Need
Students	Fair, transparent access to rooms, and to scarce equipment for study and group work.
Faculty Members	Guaranteed rooms and equipment for scheduled teaching and priority for urgent academic activity.
Administration	High utilization of rooms and of expensive equipment, with reliable usage reports.
Maintenance Staff	Ability to block an unsafe room, or a faulty equipment unit, immediately.

- Explain three genuine requirement conflicts. For each conflict, state one assumption or decision needed before design can proceed.

Conflict	Why is it genuine?	Assumption needed before design
Fair access vs. faculty priority (rooms and scarce equipment) [RL]	If faculty can always override, students may never obtain a popular room or the only portable projector; pure first come-first served may take equipment a timetabled class needs.	Timetabled teaching (room and equipment) is loaded before the semester and cannot be displaced. Urgent academic requests may displace only unconfirmed student requests. Fairness = at most 6 hours of active student bookings per week per person, counted separately for rooms and for scarce equipment.
Privacy vs. traceability	Security wants a complete record, while users expect their identity and purpose to stay private.	Every allocation is recorded in an append-only audit log, but only users with the ViewAudit permission may read identities. Other users see only free/busy status of rooms and equipment.
Utilization vs. safety	Administration wants rooms and equipment booked; maintenance and emergencies remove them without notice and invalidate confirmed bookings.	Safety always wins: a block on a room, a fault on a unit, or a building closure takes effect immediately. The system then tries to relocate the booking or substitute an equivalent unit before cancelling

- Distinguish facts given in the scenario from assumptions you introduced.

Facts stated in the scenario	Assumptions introduced (and effect on design)
Four activity types (individual, group, class, event) with different capacity, equipment and approval rules.	Classes and events need approval; individual and group requests are auto-approved if rules are met (creates a PendingApproval state).
Maintenance must block unsafe rooms immediately; security needs traceable records; users expect privacy.	Recurring reservations are limited to one semester and stored as individual reservations, so each can be disrupted separately.
Emergencies may cancel or relocate confirmed reservations; audit data must be retained and affected users notified.	During a closure, relocation order is teaching, then users with accessibility needs, then events, then the rest. The time slot is kept; changing it needs user consent.
Some equipment is movable and scarce (student condition) [RL]	Equipment is allocated by type first and a specific unit is assigned later. A booking is confirmed only when the room and all required equipment are held (all-or-nothing); optional equipment may be missing. A movable unit needs a 15-minute transfer buffer between uses in different rooms. Fixed equipment is allocated only with its own room.

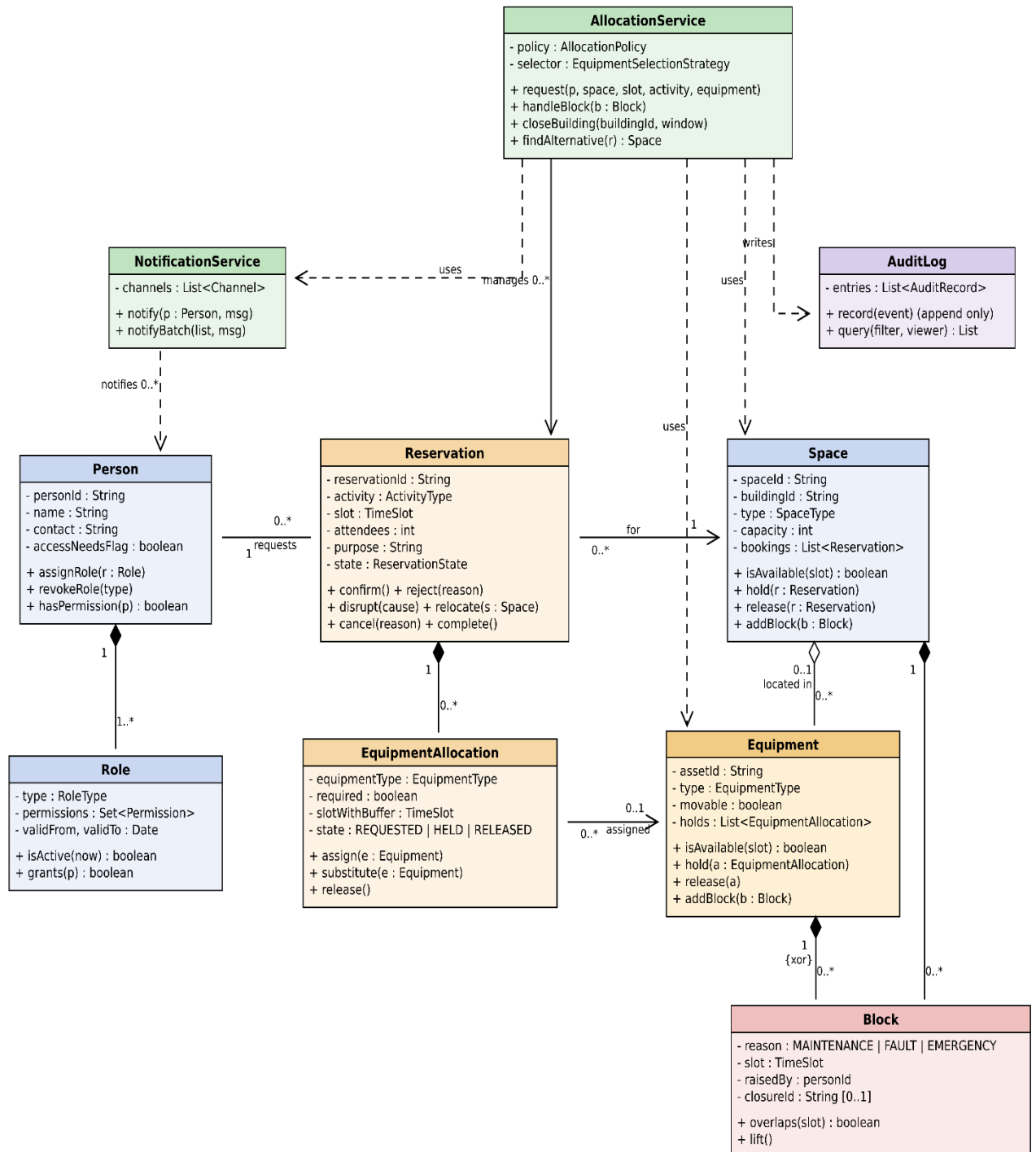
Task 2

Object Oriented Conceptual Model

- Propose 7 to 10 classes. For each class, state its responsibility and at least two relevant attributes or operations.

Class	Responsibility	Key attributes
Person	Represents a campus user and answers permission questions through its roles.	personId, name, accessNeedsFlag; assignRole(), revokeRole(), hasPermission()
Role	One role a person holds for a period, with its permissions.	type, permissions, validFrom/validTo; isActive(), grants()
Space	A bookable room; owns its own schedule and blocks and refuses overlapping holds.	buildingId, capacity, bookings; isAvailable(), hold(), release(), addBlock()
Reservation	One booking of one space for one time slot; controls its own life cycle.	activity, slot, attendees, state; confirm(), disrupt(), relocate(), cancel()
Equipment [RL]	One physical unit (e.g. portable projector, VR kit); owns its own schedule and refuses overlapping holds.	assetId, type, movable, holds; isAvailable(), hold(), release(), addBlock()
Block	A period when a space or a unit cannot be used (maintenance, fault, emergency).	reason, slot, raisedBy, closureId; overlaps(), lift()
Auditlog	Append-only record of every allocation decision; filters what each viewer may see.	entries; record(), query(filter, viewer)

- Draw a UML class diagram showing associations, multiplicities, and any justified use of composition, aggregation, or inheritance.



- Explain how your model separates responsibilities and avoids a single class controlling the entire system.

No class controls the whole system. Each rule is enforced by the object that owns the data: Space prevents double booking of rooms and Equipment prevents double booking of units because each holds its own schedule; Reservation guarantees legal state changes; Person and Role answer permission questions; AuditLog alone controls who may read identities. AllocationService only coordinates and cannot write into a schedule directly. [RL] Because equipment has its own allocation object, a faulty projector is handled by substituting one EquipmentAllocation, not by cancelling and rebooking the whole reservation.

Task 3

Abstraction, encapsulation and invariants

- Select three important classes from your model and explain what each class abstracts from the real world.

Class	What it abstracts?
Reservation	An agreement that a person may use a room at a time, with the equipment the activity needs. Keys and paper forms are ignored; only what matters for allocation, audit and relocation is kept.
Space	A physical room reduced to capacity, type and calendar; décor and exact dimensions are ignored.
Equipment [RL]	A physical unit reduced to its type, whether it can move, where it is, and when it is busy; serial numbers and purchase data are ignored.

- For each selected class, identify information that should be hidden and the operations through which it may be changed.

Class	Hidden Information	Controlled Information
Reservation	state, purpose, attendee list, list of EquipmentAllocations, history of previous spaces	confirm(), reject(), disrupt(cause), relocate(space), cancel(reason), complete(). No public setState()
Space	bookings list, blocks list	isAvailable(slot), hold(r), release(r), addBlock(b). Callers never receive the modifiable list
Equipment [RL]	holds list, blocks list, current location	isAvailable(slot), hold(a), release(a), addBlock(b), moveTo(space). Only moveTo() changes location and it refuses to move fixed units.

- Write at least four invariants or rules that must always remain true. Explain how encapsulation helps prevent invalid states.

Invariants (Always True)	How encapsulation protects it?
No two Confirmed reservations for the same space overlap in time	Only Space.hold() can add to bookings; it checks overlap and adds in one atomic step.
No equipment unit is held by two allocations whose slots, including the 15-minute transfer buffer, overlap. [RL]	Only Equipment.hold() can add to holds; it compares slotWithBuffer, which is computed inside EquipmentAllocation, not supplied by callers.
A reservation is Confirmed only while its room is held and every required EquipmentAllocation is HELD. [RL]	confirm() checks this condition; if a required unit is released, the allocation notifies its reservation, which moves to Disrupted.
Nothing is confirmed for a space or unit during an overlapping block.	hold() refuses blocked slots; addBlock() triggers disrupt() or substitute() on every overlapping booking.

Task 4

Alternative designs and decision

- Compare two ways to represent users with multiple and changing roles. One option must use inheritance and the other must use composition or role objects.

Option A: Inheritance

User is an abstract superclass with subclasses Student, Faculty, Admin, MaintenanceStaff, SecurityStaff. A student who is also a TA needs a further subclass StudentTA.

Option B: Composition (Role Objects)

A single Person class holds a collection of Role objects, each with its own permissions and validity dates

- Compare the alternatives using at least three criteria such as extensibility, coupling, duplicate behavior, multiple roles, or ease of changing permissions.

Criteria	A: Inheritance	B: Composition
Duplicate Behavior	Combination subclasses repeat permission logic from two parents (Java has no multiple class inheritance).	Permission logic lives once in Role; Person combines results.
Ease of Changing Permissions [RL]	Allowing TAs to book scarce equipment means editing subclass code and redeploying.	Add the BookScarceEquipment permission to a TA role, with an expiry date.
Coupling	Allocation code tends to use instanceof Faculty checks, coupling it to the hierarchy	Allocation code asks hasPermission(p) only, which gives low coupling.

- Choose one design for this scenario and justify the choice from the requirements.

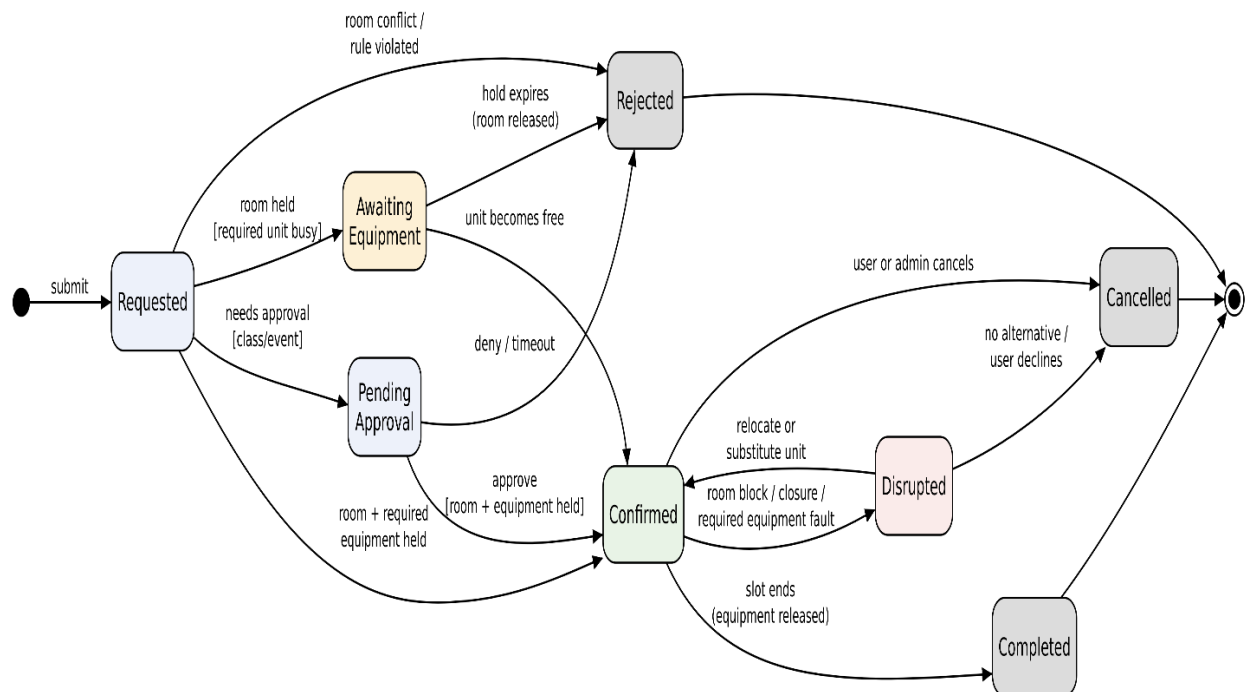
Option B

The scenario states that one person may hold several roles and that roles and permissions change during the semester, which inheritance cannot represent without recreating objects. Access to scarce equipment is a permission that the department is likely to grant and withdraw often (for example, to a TA only while a lab course runs); with role objects this is one `assignRole()` call that lapses automatically when `validTo` passes.

Task 5

Behavior and edge cases

- Show the life cycle of a reservation as a state diagram or a clearly labelled state transition table.



- Explain how your model responds to all three cases: two simultaneous requests for the same space, a maintenance block after confirmation, and an emergency cancellation or relocation.

Edge Case	How the model responds?
Two simultaneous requests for the same space (or the last unit) [RL]	AllocationService calls Space.hold(), which checks and adds in one synchronised step, so the first commit wins and the second gets a conflict and is offered findAlternative(). If both requests are for different rooms but want the last portable projector, Equipment.hold() decides the same way; the loser either proceeds without it (optional) or its room hold is rolled back (required), so no room is left held for a booking that cannot happen. Locks are always taken room first, then equipment in assetId order, to avoid deadlock.
Maintenance block after confirmation	Maintenance calls Space.addBlock(), effective immediately (A3). AllocationService.handleBlock() marks each overlapping reservation Disrupted and searches for an equivalent room. [RL] Movable units follow the booking (their allocation's buffer is re-checked for the move); fixed equipment means the new room must already have that equipment. If maintenance blocks a faulty unit instead, only that EquipmentAllocation is re-assigned with substitute() and the room booking is untouched.
Emergency cancellation or relocation	AllocationService.closeBuilding() adds an EMERGENCY block, with one closureId, to every space in the building and to every unit currently located there, because those units cannot be retrieved. Affected reservations are sorted by A7, relocated to other buildings using units outside the closed building, and cancelled if no room or required unit remains. One notifyBatch() informs users, and every audit record carries the closureId.

- Identify which object should enforce each decision and why.

Edge Case	Object Enforcements
Two simultaneous requests for the same space (or the last unit) [RL]	<i>Space</i> and <i>Equipment</i> : each owns its schedule. <i>AllocationService</i> decides priority and rollback but cannot bypass the checks.
Maintenance block after confirmation	<i>Space/Equipment</i> enforce the block; <i>Reservation</i> enforces legal transitions; <i>AllocationService</i> coordinates relocation because it sees all rooms and units.
Emergency cancellation or relocation	<i>AllocationService</i> owns relocation policy; <i>Block</i> records the scope; <i>AuditLog</i> keep communication and records separate from the decision.

Task 6

Independent exploration

Explore one concept not yet covered in detail in class: immutable value objects, the Strategy pattern, or role-based access control. In 150 to 200 words, define the concept in your own words and show one precise place where it improves your model. Use and cite at least two credible sources. Include full references.

THE STRATEGY PATTERN

The Strategy pattern places each version of an algorithm in its own class behind a shared interface, so the object that uses the algorithm can switch between versions at run time without changing its own code [1]. The client depends only on the interface, not on any particular implementation, which keeps the client stable when rules change [2]. The precise place is equipment unit selection [RL]. When several units of the requested type are free, AllocationService does not choose itself; it calls selector.select(candidates, reservation) on its field of type EquipmentSelectionStrategy. NearestUnitStrategy picks the unit closest to the booked room to reduce moving time. BalancedWearStrategy picks the least-used unit so scarce equipment ages evenly. ReserveForTeachingStrategy refuses to give students the last free unit in a teaching week. Administration can change the strategy each week without any change to AllocationService, and when the university finally defines fair access to equipment, a new strategy class is added instead of editing a long if-else chain. Each strategy can also be tested on its own.

References:

- [1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1995, pp. 315-323.
- [2] E. Freeman and E. Robson, Head First Design Patterns, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020, ch. 1.